



Terrier e Coleção Chave

Processamento e Recuperação de Informação Mestrado em Informática Médica

Braga, abril 2024

Docente: Joaquim Melo Henriques Macedo

Mariana Ribeiro, pg54061;

Mariana Almeida, pg54062;

Madalena Passos, pg54023.

Índice

Introdução.....	4
Configuração do Terrier	5
1) Relativamente à coleção do Terrier	5
a. Em que ficheiros se coloca os parâmetros que determinam o modo de funcionamento do Terrier?	5
b. Diga como se indica como se usa ou não a expansão de interrogações, a lista de stop words e o stemming?.....	5
c. Em termos do processamento das interrogações e dos documentos que variantes estão disponíveis?	6
d. Indique outras possibilidades de configuração que não foram explicitadas, embora estejam disponíveis.	7
2) Fixemos agora a nossa atenção na configuração da indexação no Terrier.....	7
a. O primeiro aspeto a parametrizar é o componente TemPipeline. Existe algum stemmer para português no Terrier? E lista de stopwords?.....	7
b. Para manter a meta-informação dos documentos existe o meta-index. O que se pode configurar aí?	8
c. Relativamente ao processo de indexação existem duas alternativas: indexação num único passo e indexação em dois passos. Quais são as diferenças fundamentais? Apresente também vantagens e desvantagens.	8
d. O que é o BlockIndexer? Podemos indexar com informação de posição? Como?	9
e. Crédito extra: Para indexação de colecções muito grandes está disponível o Hadoop Map-Reduce Indexing. Explique sucintamente.....	9
3) Vejamos como se configura a recuperação de informação.....	10
a. No modo batch temos que indicar onde o sistema deve ir buscar as interrogações. Explique como.....	10
b. Como se indica que parte do ficheiro do tópico usar para construir a interrogação?.....	10
c. Apresente quatro dos métodos usados para ponderação dos termos. Inclua pelo menos um método da família DFR. Explique sucintamente os métodos seleccionados.....	11
d. Explique os modelos de ponderação dependentes do campo.	12

e. Pode-se inflacionar a pontuação dos documentos baseados na proximidade dos termos da interrogação? Como?.....	13
f. Como se pode alterar no modo batch o formato de entrada das interrogações e saídas dos resultados?	13
g. Como incluiria na pontuação dos documentos um valor independente da interrogação como PageRank? Explique.....	14
h. Que outro tipo de configuração pode ser feita que não foi apresentada?	14
4) Explique sucintamente a linguagem de interrogação disponível no Terrier.	14
5) Crédito Extra: Explique o que é a expansão de interrogações e como está implementada no Terrier.	16
6) Avaliação	17
a. Que ferramentas tem disponível no Terrier para avaliação	17
b. Os Runs criados pelo Terrier são compatíveis com a ferramenta trec_eval. Descreva sucintamente as funcionalidades desta ferramenta. Que resultados nos permite obter?	17
Experiências com a coleção Chave	19
1) Indexe a coleção Público incluída na Chave.....	19
a. Quanto tempo demora a indexar o público com a indexação dum passo com e sem informação de posição.....	19
b. Quanto tempo demora a indexar o público com a indexação de dois passos com e sem informação de posição?	20
2) Vamos usar o Terrier com a coleção Público e o conjunto de interrogações e juízos de relevância disponibilizados no CLEF. Os resultados vão ser obtidos com 50 interrogações.....	22
a. Vamos fazer um gráfico de precisão e recall, r-precision, MAP aos 10,20 e 30.....	22
b. Vamos compara dois métodos de ponderação disponibilizados no Terrier! (por exemplo! TF-IDF e BM25) e em particular um baseado no DFR.	23
c. Qual é a melhoria dos resultados se usarmos expansão de interrogações?	26
d. O facto de se usar stopwords e stemming melhora os resultados?	26
Conclusão.....	28

Introdução

No âmbito da Unidade Curricular de Processamento foi proposta a elaboração do 1º Trabalho Teórico que aborda a utilização do pacote de software Terrier e a Coleção de Teste Chave.

O Terrier é um software de Recuperação de Informação desenvolvido na Universidade de Glasgow, Escócia. Escrito em Java, é uma ferramenta de código aberto projetada para investigação e experimentação na área, oferecendo funcionalidades de indexação e recuperação de acordo com os padrões atuais.

Este trabalho propõe a realização de uma série de perguntas teóricas acerca do funcionamento do software. No que diz respeito à recuperação de informação, são abordados métodos de ponderação de termos, modelos dependentes do campo, e formas de alterar o formato de entrada das interrogações e saídas dos resultados.

Por fim, o trabalho propõe experiências práticas com a coleção Chave, incluindo indexação da coleção Público, análise de tempo de indexação, comparação de métodos de ponderação, avaliação da expansão de interrogações e o impacto do uso de *stopwords* e *stemming* nos resultados obtidos.

Configuração do Terrier

1) Relativamente à coleção do Terrier

a. Em que ficheiros se coloca os parâmetros que determinam o modo de funcionamento do Terrier?

Inicialmente, após a instalação do Terrier, na diretoria *etc*, encontra-se presente o ficheiro *terrier.properties.sample* e o ficheiro *logback.xml*. O primeiro serve como um modelo, contendo uma estrutura básica de configurações possíveis para o Terrier. Quanto ao ficheiro *logback.xml*, este é normalmente utilizado para configurar o sistema de *logging*.

Posteriormente, após a configuração inicial do Terrier, foi gerado um novo ficheiro denominado *terrier.properties* na mesma diretoria através do comando *terrier_setup.sh* (Figura 1).

```
maryy@Mariana:/mnt/c/Users/maryy/terrier-project-5.5$ ./bin/trec_setup.sh /mnt/c/users/maryy/Terrier/Cole_Pub
Creating collection.spec file.
Creating jforests.properties file.
Creating features.list file.
Creating logging configuration (logback.xml) file in /mnt/c/Users/maryy/terrier-project-5.5/etc/
Creating terrier.properties file.
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951221.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951222.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951223.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951224.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951226.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951227.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951228.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951229.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951230.sgml
/mnt/c/users/maryy/Terrier/Cole_Pub/publico95/ED951231.sgml
Updated collection.spec file (727 lines). Please check that it contains
all and only all the files to be indexed, or create it manually.
```

Figura 1- *trec_setup.sh*.

Este ficheiro é onde se colocam os parâmetros, no formato chave-valor, que determinam o modo do funcionamento do Terrier, contendo especificações acerca de configurações do índice, caminhos para ficheiros de configuração, configurações específicas para diferentes tipos de coleções, como *tags* de documentos e codificação padrão, etc.

b. Diga como se indica como se usa ou não a expansão de interrogações, a lista de stop words e o stemming?

A expansão de interrogações, assim como a lista de *stop words* e *stemming* são utilizados em Terrier recorrendo a edições do ficheiro *terrier.properties*, alterando determinadas configurações.

Para a expansão de interrogações, poderá ser ativada a opção *QueryExpansion* em dois conjuntos de parâmetros distintos *querying.default.controls* e *querying.allowed.controls*, assim como *querying.postprocesses.order* e *querying.postprocesses.controls*, dependendo se é pretendido fornecer a opção de aplicar essa configuração ao utilizador ou integrá-la no processo de tratamento da consulta. Adicionalmente, podem ser utilizadas configurações adicionais como *expansion.documents* e *expansion.terms* para referir o número de documentos e termos a serem usados.

Relativamente à utilização de *stop words*, o *Terrier* permite especificar um documento contendo uma lista de palavras mais comuns a filtrar nas consultas através da configuração *stopwords.filename* no arquivo *terrier.properties*. Caso não se pretenda a definição de *stop words* pode-se deixar este documento em branco.

Por fim, a utilização de *stemming* pode ser efetuada configurando e habilitando o algoritmo *PorterStemmer*. Se não for pretendida a utilização desta etapa, deverá definir-se *trec.stemming=false*.

c. Em termos do processamento das interrogações e dos documentos que variantes estão disponíveis?

A expansão de interrogações, a lista de *stop words* e o *stemming* são elementos-chave que afetam diretamente o processo de recuperação de informações. ^[1]

Expansão de Interrogações

A primeira técnica mencionada faz com que termos adicionais sejam automaticamente adicionados à interrogação original, melhorando então, os resultados de uma procura. Estes termos adicionais correspondem a termos que aparecem com grande frequência em documentos nas diferentes etapas da procura. Os documentos e termos são escolhidos segundo algoritmos de ponderação de peso, isto é, consoante uma importância atribuída.

O software *Terrier* utiliza alguns destes modelos, como por exemplo, BB2 (Bose-Einstein Model), IFB2 (Inverse Term Frequency model with Bernoulli after-effect and normalisation 2), PL2(Poisson model with Laplace), entre outros. É de notar que estes modelos se incluem nos modelos de framework DFR – Divergence from Randomness.

No geral, para processar as queries, é, inicialmente, necessário assegurar que o ficheiro *topics* está bem especificado na propriedade *trec.topics*. Posteriormente, utiliza-se o comando `bin/trec_terrier.sh -r -c 1.0` (o parâmetro *-c* especifica o valor do parâmetro para a normalização da frequência de termos).

Para utilizar um modelo de ponderação de peso específico, é necessário especificar a propriedade *trec.model*, utilizando o seguinte comando: `bin/trec_terrier.sh -r -Dtrec.model=DLH13`.

Lista de Stop Words

As Stop Words são palavras comuns na língua que geralmente devem ser ignoradas durante o processamento de uma procura, como, por exemplo, “e”, “para” e “com”. Uma vez que são

palavras comuns, ocorrem com grande frequência em vários documentos, e por isso não são consideradas comuns no processo de recuperação de informação. O software Terrier permite configurar a lista de stop words utilizada através da configuração da propriedade `termPipeline`, dando acesso à lista de stop words de vários idiomas.

Stemming

O processo de Stemming permite reduzir as palavras às suas formas básicas ou raízes, através da remoção de sufixos e prefixos, permitindo uma melhor correspondência na consulta de palavras nos documentos. No software Terrier, o processo de stemming é aplicado pelo componente chamado “PorterStemmer”, que utiliza o algoritmo de Stemming de Porter para a língua inglesa.

Após a tokenização e remoção de stopwords, o algoritmo de Stemming de Porter é aplicado às palavras restantes. Este algoritmo é um dos algoritmos de stemming mais populares e amplamente utilizados para a língua inglesa. Para a língua portuguesa pode ser usado o algoritmo “PortugueseSnowBall”.

Estes processos acontecem através da estrutura “TermPipeline” configurada em “terrier.properties”, onde o *indexer* itera sobre os documentos, enviando para o pipeline cada termo encontrado.

d. Indique outras possibilidades de configuração que não foram explicitadas, embora estejam disponíveis.

Uma outra configuração disponível está relacionada com a possibilidade de registrar a frequência com que os termos ocorrem em vários campos dos documentos. Desta forma, devem ser especificados na propriedade `FieldTags.process`, os campos necessários. Por exemplo, para registrar quando um termo ocorre nas *tags* TITLE ou H1 HTML de um documento, define-se `FieldTags.process=TITLE,H1`. É de notar que as `FieldTags` são case-insensitive, isto é, não diferenciam maiúsculas de minúsculas.

2) Fixemos agora a nossa atenção na configuração da indexação no Terrier.

a. O primeiro aspeto a parametrizar é o componente `TemPipeline`. Existe algum stemmer para português no Terrier? E lista de stopwords?

Relativamente ao *stemmer* para a língua portuguesa, o programa Terrier suporta o algoritmo *PortugueseSnowballStemmer*.

Quanto à lista de stop words para o português, o programa não possui uma lista de *stop words* para este idioma. No entanto, esta pode ser criada, sendo especificada no arquivo *terrier.properties* usando o parâmetro *stopwords.filename*, como mencionado anteriormente.

b. Para manter a meta-informação dos documentos existe o meta-index. O que se pode configurar aí?

Na configuração de meta-informação são definidos alguns parâmetros quanto à indexação dos documentos, no ficheiro *terrier.properties*.

- **indexer.meta.forward.keys:** Esta propriedade permite construir a lista de atributos de documentos de forma a ser guardada no meta-index. A título de exemplo pode atribuir-se "`indexer.meta.forward.keys=docno`", sendo "docno" um identificador único para cada documento na coleção.

- **indexer.meta.forward.keylens:** Nesta propriedade pode ser especificado o comprimento máximo da chave (key) no meta-index.

- **indexer.meta.reverse.keys:** De acordo com a documentação, esta propriedade indica uma Lista delimitada por vírgulas de atributos que denotam exclusivamente um documento. Isso significa que, dado um valor de um atributo, um único documento pode ser identificado.

c. Relativamente ao processo de indexação existem duas alternativas: indexação num único passo e indexação em dois passos. Quais são as diferenças fundamentais? Apresente também vantagens e desvantagens.

Em relação ao processo de indexação, existem duas alternativas: indexação num único passo e indexação em dois passos.

A indexação em dois passos, também conhecida como indexação clássica, envolve a criação de índice direto e, de seguida, a inversão dessa estrutura de dados para a construção um índice invertido. Esta abordagem é mais lenta, mas tem como vantagem a construção de um índice direto que pode ser utilizado para expansão de interrogações. No entanto, esta indexação não é considerada escalável para grandes coleções de documentos, sendo que o máximo é de cerca de 25 milhões de documentos. Em termos práticos, para execução da indexação em dois passos, é necessário utilizar o comando `batchindexing`.

Relativamente à indexação num único passo, esta é implementada de forma mais rápida. Ao contrário da indexação em dois passos onde se constrói um ficheiro direto a partir da coleção, as *posting lists* de termos são mantidas na memória e escritas no disco quando a memória está cheia. De seguida, ocorre a fusão dos ficheiros temporários para formar o léxico e o ficheiro invertido.

Desta forma, a principal vantagem consiste na rapidez de indexação, embora incorra na ausência de um índice direto, que pode ser útil para a expansão de interrogações. No entanto, este índice pode ser criado posteriormente, usando o comando `inverted2direct` do Terrier. . Em termos práticos, para execução da indexação num único passo, é necessário utilizar o comando `batchindexing -j`.

d. O que é o `BlockIndexer`? Podemos indexar com informação de posição? Como?

O `BlockIndexer` resume-se como parte do processo de indexação do Terrier, responsável por indexar os documentos em blocos paralelamente, de modo a tornar o processo mais eficiente.

Relativamente à indexação com informação de posição, esta permite manter informações sobre a posição exata de um termo dentro de um documento, o que pode ser útil em várias aplicações e isto pode ser efetuado habilitando a opção de indexação de posição, seguida da configuração da etapa de processamento de texto apropriada para capturar essas informações. Isto resume-se, então, à adição de `indexer.meta.forward.keys=position` ao ficheiro `terrier.properties` e à aplicação de um `tokenizador` para capturar as informações de posição.

e. Crédito extra: Para indexação de colecções muito grandes está disponível o Hadoop Map-Reduce Indexing. Explique sucintamente.

O algoritmo de *Hadoop MapReduce* é uma framework de software que permite o processamento de quantidades muito grandes de dados em clusters paralelos, milhares de nodos. [2,3]

O processamento eficaz de colecções de documentos muito grandes permite associar o indexador de passo único em secções das colecções a várias tarefas do mapa. Das tarefas mencionadas podem resultar, três possíveis resultados: termos e posting-lists de pequena dimensão, índices de documentos para cada tarefa do mapa, e ainda o número de documentos guardados por execução.

No software Terrier, para se poder utilizar *MapReduce* na indexação é necessário utilizar-se o cluster *Hadoop*. Assim, nas properties pode configurar-se a propriedade `trec.collection.class` associada a um construtor `InputStream`.

Este algoritmo pode ser utilizado de duas formas diferentes: através do termo-particionado ou do documento-particionado. No primeiro, é criado um índice único, facilitando a recuperação de informação, sendo o índice invertido dividido em 26 ficheiros. Na segunda opção, são criados vários índices de acordo com o número de cada redutor, sendo esta opção mais eficaz para casos em que o *corpus* é de dimensões superiores às suportadas por um índice apenas.

Para executar o indexador MapReduce pode utilizar-se o seguinte comando: `bin/trec_terrier.sh -i -H`.

3) Vejamos como se configura a recuperação de informação.

a. No modo batch temos que indicar onde o sistema deve ir buscar as interrogações. Explique como.

No modo *batch*, é necessário indicar ao sistema onde encontrar as interrogações que serão usadas para recuperar os documentos. Isto é efetuado no ficheiro *terrier.properties* através da especificação de um ficheiro de interrogações ou de uma diretoria que contém os respetivos ficheiros de queries (Figura 2).

```
#the list of query files to process are normally listed in the file etc/trec.topics.list
trec.topics=/mnt/c/Users/maryy/terrier-project-5.5/topics/topics.txt
```

Figura 2-Configuração do caminho para ficheiro de queries em *terrier.properties*.

b. Como se indica que parte do ficheiro do tópico usar para construir a interrogação?

Para indicar as partes do ficheiro a utilizar para construir a interrogação, são definidas *tags* indicando quais os elementos a considerar para esta. Algumas destas *tags* estão associadas à *tag* que inicia a consulta no arquivo (*TrecQueryTags.doctag*), quais as partes desta a serem consideradas (*TrecQueryTags.process*) ou ignoradas (*TrecQueryTags.skip*), ao identificador único da consulta (*TrecQueryTags.idtag*), entre outras.

```
#query tags specification
# The tags specified by TrecQueryTags are case-insensitive

# short (s): title only, <PT-title>
# medium (m): Title and description, <PT-title> and <PT-desc>
# large (l): all information, <PT-title>, <PT-desc> and </PT-narr>

TrecQueryTags.doctag=TOP
TrecQueryTags.process=TOP,NUM,PT-TITLE,PT-DESC
TrecQueryTags.idtag=NUM
TrecQueryTags.skip=PT-NARR
```

Figura 3- Configurações para a construção de interrogações.

c. Apresente quatro dos métodos usados para ponderação dos termos. Inclua pelo menos um método da família DFR. Explique sucintamente os métodos selecionados.

Em sistemas de *Information Retrieval*, a eficácia em encontrar e classificar documentos relevantes em resposta a consultas dos utilizadores é crucial. Uma das etapas fundamentais nesse processo é a ponderação dos termos, onde os termos nos documentos e nas consultas são atribuídos pesos com base em sua importância para a relevância dos documentos. Essa ponderação influencia diretamente na classificação e na ordenação dos resultados da pesquisa. No Terrier existem alguns exemplos de métodos para ponderação de termos, nomeadamente, BM25(Best Matching 25), TF_IDF (Term Frequency-Inverse Document Frequency), PL2 (Poisson-Laplace 2), e DPH, sendo o que os dois primeiros pertencem à família de modelos IDF e os restantes à família de modelos DFR. Estes modelos são utilizados através da *package* “org.terrier.matching.models.WeightingMidel”

BM25

O BM25 é um modelo probabilístico amplamente utilizado em sistemas de recuperação de informação que calcula a pontuação de relevância de um documento com base na frequência do termo na consulta e no documento, bem como na frequência inversa do termo na coleção. O BM25 considera fatores como a saturação do termo e a relevância do documento para a consulta. De forma mais detalhada, os parâmetros $k_1 = 1.2d$, $k_3 = 8d$ e $b = 0.75d$ por defeito.

TF IDF

TF_IDF, (Term Frequency-Inverse Document Frequency), é um modelo clássico de ponderação de termos que atribui pesos aos termos com base em sua frequência no documento e em quão exclusivos eles são na coleção. Os termos que aparecem com mais frequência no documento, mas menos frequentemente na coleção, recebem uma pontuação mais alta.

De forma mais detalhada, a frequência do termo é calculada com o número de vezes que este aparece no documento em relação ao número total de palavras do documento. Por outro lado, a Frequência Inversa de Documentos é dada através da aplicação do logaritmo no número de documentos em relação ao número de documentos onde o termo pretendido é encontrado.

PL2

O PL2, (Poisson-Laplace 2), é um modelo da família DFR que utiliza a estimação de Poisson para aleatoriedade e a sucessão de Laplace para normalização. Este método calcula a pontuação de relevância de um documento considerando a frequência do termo no documento, a frequência do termo na coleção e a normalização. O objetivo final deste tipo de ponderação é ter um modelo sem parâmetros ajustáveis, não considerando a relevância do documento na pesquisa, mas dando importância ao ganho calculado na recuperação de um documento onde consta o termo.

DPH

De acordo com a documentação este modelo implementa um modelo de ponderação hipergeométrica DPH, onde o "P" representa a normalização de Popper. Este é um modelo de ponderação sem parâmetros. Mesmo que o usuário especifique um valor de parâmetro, isso não afetará os resultados. É altamente recomendado utilizar o modelo com expansão de consulta.

d. Explique os modelos de ponderação dependentes do campo.

Atualmente, o Terrier suporta modelos de ponderação dependentes do campo. Estes modelos consideram a presença e frequência de ocorrência de um termo num dado campo. Os principais modelos são os seguintes:

- **PL2F**: modelo de normalização por campo baseado em PL2.
- **BM25F**: modelo de normalização por campo baseado em BM25.
- **ML2**: modelo multinomial baseado em campos.
- **MDL2**: modelo multinomial baseado em campos onde o multinomial é substituído por uma aproximação.

Para o uso de um modelo de ponderação dependentes do campo, é necessário indexar usando campos. Geralmente, modelos dependentes campos diferentes podem ter diferentes parâmetros, que, por sua vez, podem ser controlados por várias propriedades. Para modelos como PL2F e BM25F, são requeridos parâmetros de normalização para cada campo, como c.0, c.1, etc, sendo necessário a execução do seguinte comando:

```
bin/terrier batchretrieval -w PL2F -Dc.0=1.0 -Dc.1=2.3 -Dc.3=40 -  
Dw.0=4 -Dw.1=2 -Dw.3=25
```

Com o intuito de melhorar a eficiência dos modelos de ponderação baseados em campos, é recomendado alterar manualmente o ficheiro *data.properties* do índice criado de forma a alterar implementação do *DocumentIndex* em uso. Assim, esta propriedade deverá ser atualizada da seguinte forma:

```
index.document.class=org.terrier.structures.FSAFieldDocumentIndex.  
x.
```

e. Pode-se inflacionar a pontuação dos documentos baseados na proximidade dos termos da interrogação? Como?

Sim, pode-se inflacionar a pontuação dos documentos com base na proximidade dos termos da interrogação, recorrendo a modelos de dependência que atribuem pontuação mais alta para documentos onde os termos se encontram mais próximos.

Existem modelos que utilizam, para este processo, a proximidade dos termos (*DFRDependenceScoreModifier*), padrões de co-ocorrência significativos (*MRFDependenceScoreModifier*), a proximidade e ordenação dos termos (*IFB2DependenceScoreModifier*), a proximidade dos termos em campos relevantes (*BM25FDependenceScoreModifier*), entre outros.

f. Como se pode alterar no modo batch o formato de entrada das interrogações e saídas dos resultados?

De acordo com a documentação, a principal opção para a extração em *batch retrieval* é através de TRECQuerying, podendo ser extendida das seguintes formas:

- Formato dos tópicos de entrada: O Terrier suporta tópicos em dois formatos (formato etiquetado TREC ou uma consulta por linha – “one query per line”). Se nenhum desses formatos for adequado, então pode ser implementado outro *QuerySource* que consiga como analisar o ficheiro *topics*. Para configurar Terrier para usar seu novo *QuerySource* pode usar-se a propriedade `trec.topics.parser`. Por exemplo, `trec.topics.parser = my.package.DBTopicsSource`. Em modo *batch* pode ser usado o seguinte comando para usar-se por exemplo o *SingleLineTRECQuery*: `terrier batchretrieval -s`

- Formato dos resultados de saída: Pode ser implementado outro *OutputFormat* para alterar o formato dos resultados nos arquivos com extensão “.res”. Utilizando a propriedade `trec.querying.outputformat` pode configurar-se o Terrier para usar o formato *OutputFormat*. Por exemplo, `trec.querying.outputformat = my.package.MyTRECResultsFormat`. Em modo *batch*, pode ser usado `terrier batchretrieval -F my.package.some.other.OutputFormat`.

g. Como incluiria na pontuação dos documentos um valor independente da interrogação como PageRank? Explique.

O *PageRank* é um algoritmo usado pelo motor de busca do Google para ordenar as páginas da web. Este algoritmo atribui uma pontuação numérica a cada página, dependendo do número e qualidade dos links que a referenciam. Assim, quanto mais links de qualidade uma página tiver, maior será a pontuação de *PageRank*, afetando a posição dos resultados de pesquisa.^[4]

O Terrier pode facilmente incluir na pontuação dos documentos um valor independente da interrogação no modelo de recuperação. A maneira mais simples de o fazer é com recurso ao *SimpleStaticScoreModifier*. Esta *feature* pode ser implementada da seguinte forma:

```
bin/terrier batchretrieval Dmatching.dsms =SimpleStaticScoreMo  
difier Dssa .input.file= /path/to/feature -Dssa.input.type=li  
stofscores -Dssa.w=0.5.
```

h. Que outro tipo de configuração pode ser feita que não foi apresentada?

Alguns exemplos de configurações adicionais que podem ser feitas no processo de recuperação de informação são:

- *trec.output.format.length*

Esta configuração permite controlar o número máximo de documentos incluídos nos resultados de saída. Esta configuração é importante para garantir que todos os resultados relevantes sejam apresentados adequadamente, especialmente em consultas que retornam muitos documentos.

- *ignore.low.idf.terms*

Esta configuração é relevante para controlar se os termos da consulta com frequência de documento baixa devem ser ignorados durante a recuperação, pelo que termos com frequência de documento alta podem ser menos informativos e podem afetar negativamente a precisão da pesquisa.

4) Explique sucintamente a linguagem de interrogação disponível no Terrier.

O Terrier oferece duas linguagens de consulta - uma linguagem de consulta de alto nível voltada para o utilizador e uma linguagem de consulta de baixo nível para desenvolvedores, que é expressa em termos de operações de correspondência (matching ops). Todas as consultas do

utilizador são reescritas em operações de correspondência. De acordo com a documentação, a linguagem de consulta com operações de correspondência é inspirada nas linguagens de consulta do Indri e do Galago. O Terrier oferece uma linguagem de consulta flexível para os utilizadores, permitindo procurar com frases, campos ou especificar que os termos são obrigatórios para aparecer nos documentos recuperados.

Alguns exemplos de linguagem de consulta para o utilizador são:

- **termo1 termo2** - recupera documentos que contenham termo1 ou termo2 (não precisando de conter ambos).
- **{termo1 termo2}** - recupera documentos que contenham termo1 ou termo2, onde eles são tratados como sinónimos um do outro (não precisando de conter ambos).
- **termo1^{2.3}** - o peso de termo1 é multiplicado por 2.3.
- **+termo1 +termo2** - recupera documentos que devem conter termo1 e devem conter termo2.
- **+termo1 -termo2** - recupera documentos que devem conter termo1 e não devem conter termo2.
- **+title:termo1** - recupera documentos que contenham termo1 no campo de título.
- **title:(termo1 termo2)^{2.3}** - recupera documentos que contenham termo1 ou termo2 no campo de título e aumenta seus pesos em 2.3.
- **-title:term2** - recupera documentos que contenham termo1, mas não devem conter termo2 no campo de título.
- **"termo1 termo2"** - recupera documentos onde os termos termo1 e termo2 aparecem numa frase.
- **"term1 term2"~n** - recupera documentos onde os termos termo1 e termo2 aparecem dentro de uma distância de n blocos, sendo que ordem dos termos não é considerada.

Quanto às consultas de correspondência, existem dois tipos de operadores: *semânticos*, no sentido que causam a geração de uma lista de postagens com semânticas específicas no momento da correspondência; em contraste, os operadores *sintáticos* que permitem que os atributos dos operadores semânticos sejam alterados. Alguns exemplos de operadores semânticos são:

- **termo1** - pontua documentos que contenham este único termo de consulta.
- **termo1.title** - pontua documentos que contenham este único termo de consulta no título.
- **#band(op1 op2)** - pontua um termo contendo ambos os operadores. A frequência de cada documento correspondente é 1.
- **#syn(op1 op2)** - pontua documentos que contenham ou op1 ou op2. A frequência de cada documento correspondente é a soma das frequências das palavras constituintes.

- **#prefix(termo1)** - pontua documentos que contenham termos prefixados por termo1.
- **#fuzzy(termo1)** - pontua documentos que contenham termos que correspondam de forma difusa a termo1. Veja FuzzyTermOp para mais informações.
- **#uw8(op1 op2)** - o operador #uwN pontua documentos op1 ou op2 dentro de janelas desordenadas de N tokens - neste caso, janelas de tamanho 8 tokens.
- **#1(op1 op2)** - o operador #1 pontua documentos op1 ou op2 aparecendo adjacente.
- **#band(op1 op2)** - o operador #band pontua documentos que contenham tanto op1 quanto op2.
- **#base64(termo1)** - permite uma representação base64 de um termo de consulta ser expressa que não é diretamente compatível com o *matchop ql*.

Quanto aos dois operadores sintáticos, estes encontram-se explicados a seguir

- **#tag(tagName op1 op2)** - define o atributo de tag dos termos de consulta.
- **#combine:k=v(op1 op2)** - o operador #combine permite agrupar vários operadores juntos. Além disso, múltiplos pares chave-valor podem ser especificados para controlar estes termos de consulta. Em particular, o peso (frequência do termo de consulta) de um termo pode ser controlado configurando o índice desse operador - por exemplo, #combine:0=2:1=1(op1 op2) definirá o dobro de peso em op1 em comparação com op2. O modelo de ponderação (wmodel) e a tag (tag) do(s) termo(s) de consulta também podem ser definidos, por exemplo, #combine:tag=segundo:wmodel=PL2(op1 op2).

5) Crédito Extra: Explique o que é a expansão de interrogações e como está implementada no Terrier.

A expansão de interrogações é uma técnica utilizada para melhorar a qualidade dos resultados obtidos a partir de uma pesquisa feita por um utilizador. Assim, pretende-se reutilizar os resultados da primeira pesquisa para ajustar a interrogação, conduzindo assim a uma nova pesquisa com melhores resultados.

No Terrier, o mecanismo de expansão de interrogações extrai os termos mais informativos dos documentos mais relevantes retornados como os termos expandidos da *query*. Durante este processo, os termos nos documentos mais relevantes são ponderados utilizando um modelo específico de ponderação de termos do DFR (Divergência da Aleatoriedade). Atualmente, o Terrier implementa os modelos de ponderação de termos Bo1 (Bose-Einstein 1), Bo2 (Bose-Einstein 2) e KL (Kullback-Leibler), sendo que estes seguem, por padrão, uma abordagem sem parâmetros.

Uma alternativa a este método é o mecanismo de expansão de interrogações de Rocchio. Neste mecanismo, os utilizadores podem ajustar o parâmetro `parameter.free.expansion` para falso no ficheiro `terrier.properties`. Adicionalmente, denotam-se ainda dois parâmetros que podem ser especificados na expansão de interrogações, sendo estes o número de documentos mais retornados, `expansion.documents` (3 por *default*) e o número de termos mais informativos retirados desses documentos, configurado pela propriedade `expansion.terms` (10 por *default*).

6) Avaliação

a. Que ferramentas tem disponível no Terrier para avaliação

O Terrier dispõe de algumas ferramentas de avaliação, como o *TREC Eval* e o *Terrier Interactive*. Estas ferramentas permitem calcular métricas como a precisão, o *recall*, *F1 score*, entre outras e apresentar os resultados numa interface interativa, útil para análise qualitativa dos resultados e depuração de consultas, respetivamente.

b. Os Runs criados pelo Terrier são compatíveis com a ferramenta `trec_eval`. Descreva sucintamente as funcionalidades desta ferramenta. Que resultados nos permite obter?

O Terrier utiliza o *jtreceval* para fornecer a avaliação de execuções de recuperação padrão. O *jtreceval* contém o *trec_eval* compilado para ser executado em várias plataformas padrão (por exemplo, Windows/Linux/Mac x86). Para plataformas não suportadas, o Terrier também inclui um pacote de avaliação Java legado para avaliar os resultados das tarefas TREC *ad hoc* e de localização de páginas nomeadas.

Para executar uma avaliação, é necessário inicialmente especificar o arquivo de avaliações de relevância na propriedade `trec.qrels`. Para avaliar todos os arquivos de resultados “.res” na pasta `var/results`, pode utilizar-se o seguinte comando:

```
bin/terrier batchevaluate
```

O comando *batchevaluate* avalia cada arquivo “.res” na pasta `var/results` usando o *trec_eval*. Para avaliar um arquivo de resultado específico dando o nome do arquivo, pode usar-se o comando:

```
bin/terrier batchevaluate -e InL2c1.0_0.res
```

Ou ainda:

```
bin/trec_terrier.sh -e ./var/results/InL2c1.0_0.res
```

É de notar que o comando acima avalia apenas `./var/results/InL2c1.0_0.res`. Para um arquivo de resultado nomeado “`x.res`”, o resultado da avaliação é guardado no arquivo “`x.eval`”, que contém o conteúdo conforme mostrado no seguinte exemplo:

runid	all	InL2c1.0	iprec_at_recall_0.50	all	0.2710
num_q	all	93	iprec_at_recall_0.60	all	0.2028
num_ret	all	91930	iprec_at_recall_0.70	all	0.1555
num_rel	all	2083	iprec_at_recall_0.80	all	0.1075
num_rel_ret	all	1941	iprec_at_recall_0.90	all	0.0607
map	all	0.2948	iprec_at_recall_1.00	all	0.0245
gm_map	all	0.1980	P_5	all	0.4667
Rprec	all	0.2998	P_10	all	0.3677
bpref	all	0.9359	P_15	all	0.3097
recip_rank	all	0.7134	P_20	all	0.2747
iprec_at_recall_0.00	all	0.7376	P_30	all	0.2394
iprec_at_recall_0.10	all	0.6431	P_100	all	0.1285
iprec_at_recall_0.20	all	0.5232	P_200	all	0.0789
iprec_at_recall_0.30	all	0.4079	P_500	all	0.0382
iprec_at_recall_0.40	all	0.3424	P_1000	all	0.0209

As medidas de avaliação exibidas acima são médias ao longo de um conjunto de consultas. Pode obter-se resultados por consulta usando a opção `-p` na linha de comando:

```
bin/terrier batchevaluate -e PL2c1.0_0.res -p
```

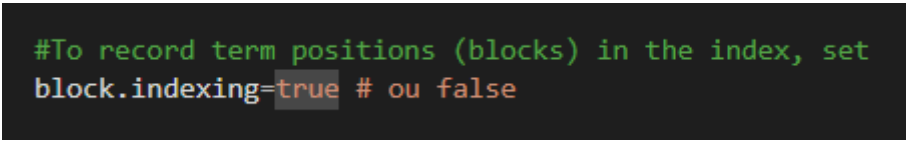
O resultado de saída resultante guardado no ficheiro “`.eval`” correspondente conterá mais resultados, com a coluna do central indicando o ID da consulta.

Experiências com a coleção Chave

1) Indexe a coleção Público incluída na Chave

a. Quanto tempo demora a indexar o público com a indexação dum passo com e sem informação de posição.

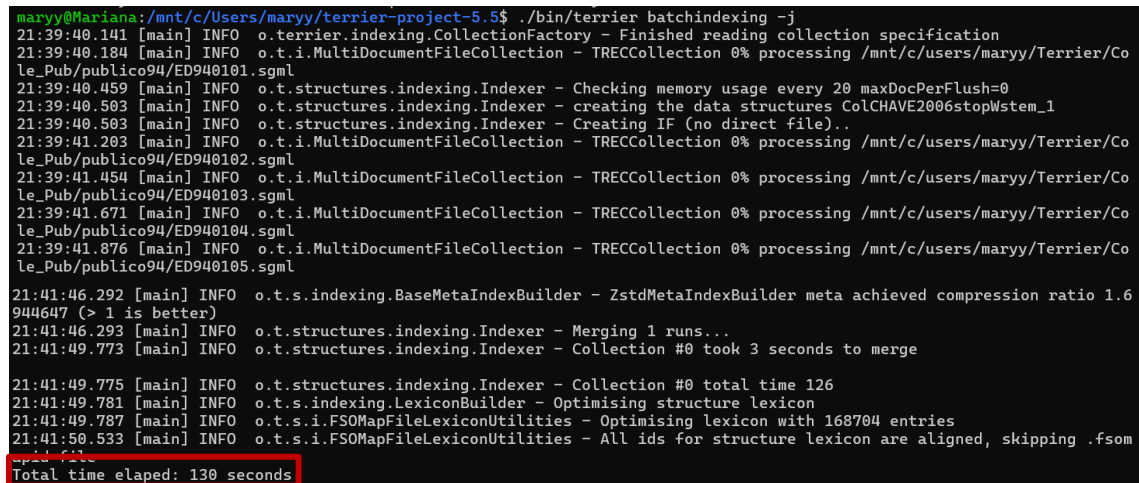
O software Terrier permite fazer o processo de indexação com ou sem informação de posição através da configuração *block.indexing* no ficheiro *terrier.properties*. Para poder indexar com informação de posição é necessário atribuir o valor *True*, como mostra a Figura 4.



```
#To record term positions (blocks) in the index, set
block.indexing=true # ou false
```

Figura 4-Configuração da propriedade block.indexing.

Para visualizar o intervalo de tempo necessário para esta indexação é necessário utilizar o comando `bin/terrier batchindexing -j`, como se pode verificar na Figura 5. Neste caso, todo o processo durou cerca de 130 segundos.



```
maryy@Mariana: /mnt/c/Users/maryy/terrier-project-5.5$ ./bin/terrier batchindexing -j
21:39:40.141 [main] INFO o.terrier.indexing.CollectionFactory - Finished reading collection specification
21:39:40.184 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940101.sgm
21:39:40.459 [main] INFO o.t.structures.indexing.Indexer - Checking memory usage every 20 maxDocPerFlush=0
21:39:40.503 [main] INFO o.t.structures.indexing.Indexer - creating the data structures ColCHAVE2006stopWstem_1
21:39:40.503 [main] INFO o.t.structures.indexing.Indexer - Creating IF (no direct file)...
21:39:41.203 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940102.sgm
21:39:41.454 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940103.sgm
21:39:41.671 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940104.sgm
21:39:41.876 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940105.sgm
21:41:46.292 [main] INFO o.t.s.indexing.BaseMetaIndexBuilder - ZstdMetaIndexBuilder meta achieved compression ratio 1.6
904647 (> 1 is better)
21:41:46.293 [main] INFO o.t.structures.indexing.Indexer - Merging 1 runs...
21:41:49.773 [main] INFO o.t.structures.indexing.Indexer - Collection #0 took 3 seconds to merge
21:41:49.775 [main] INFO o.t.structures.indexing.Indexer - Collection #0 total time 126
21:41:49.781 [main] INFO o.t.s.indexing.LexiconBuilder - Optimising structure lexicon
21:41:49.787 [main] INFO o.t.s.i.FSOMapFileLexiconUtilities - Optimising lexicon with 168704 entries
21:41:50.533 [main] INFO o.t.s.i.FSOMapFileLexiconUtilities - All ids for structure lexicon are aligned, skipping .fsom
map file
Total time elapsed: 130 seconds
```

Figura 5-Processo de indexação de passo único com informação de posição.

De forma contrária, é possível realizar o processo de indexação sem informação de posição alterando a propriedade *block.indexing* para *False*. O processo de indexação encontra-se abaixo e é conseguido novamente com o comando `bin/terrier batchindexing -j`:

```
maryy@Mariana:/mnt/c/Users/maryy/terrier-project-5.5$ ./bin/terrier batchindexing -j
21:46:50.098 [main] INFO o.terrier.indexing.CollectionFactory - Finished reading collection specification
21:46:50.142 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940101.sgml
21:46:50.424 [main] INFO o.t.structures.indexing.Indexer - Checking memory usage every 20 maxDocPerFlush=0
21:46:50.462 [main] INFO o.t.structures.indexing.Indexer - creating the data structures ColCHAVE2006stopWstem_1
21:46:50.462 [main] INFO o.t.structures.indexing.Indexer - Creating IF (no direct file)..
21:46:51.114 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940102.sgml
21:46:51.384 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940103.sgml
21:46:51.613 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940104.sgml
21:46:51.874 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940105.sgml
21:48:53.011 [main] INFO o.t.s.indexing.BaseMetaIndexBuilder - ZstdMetaIndexBuilder meta achieved compression ratio 1.6
944647 (> 1 is better)
21:48:53.011 [main] INFO o.t.structures.indexing.Indexer - Merging 1 runs...
21:48:56.240 [main] INFO o.t.structures.indexing.Indexer - Collection #0 took 3 seconds to merge
21:48:56.242 [main] INFO o.t.structures.indexing.Indexer - Collection #0 total time 123
21:48:56.249 [main] INFO o.t.s.indexing.LexiconBuilder - Optimising structure lexicon
21:48:56.256 [main] INFO o.t.s.i.FSOMapFileLexiconUtilities - Optimising lexicon with 168704 entries
21:48:56.957 [main] INFO o.t.s.i.FSOMapFileLexiconUtilities - All ids for structure lexicon are aligned, skipping .fsom
...
Total time elapsed: 127 seconds
```

Figura 6-Processo de indexação de passo único sem informação de posição.

Para este último caso, o processo foi terminado em 127 segundos, sendo 3 segundos mais rápido do que o primeiro caso.

b. Quanto tempo demora a indexar o público com a indexação de dois passos com e sem informação de posição?

Para poder averiguar o tempo de execução da indexação de dois passos com informação de posição é necessário voltar a configurar no ficheiro *terrier.properties* a propriedade *block.indexing* com valor *True*. Neste caso, para assegurar que está a ser usada a indexação de dois passos é necessário usar o comando `bin/terrier batchindexing` como mostra a Figura 7.

```
maryy@Mariana:/mnt/c/Users/maryy/terrier-project-5.5$ ./bin/terrier batchindexing
21:59:26.223 [main] INFO o.terrier.indexing.CollectionFactory - Finished reading collection specification
21:59:26.267 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940101.sgml
21:59:26.551 [main] INFO o.t.structures.indexing.Indexer - creating the data structures ColCHAVE2006stopWstem_1
21:59:26.552 [main] INFO o.t.structures.indexing.Indexer - BlockIndexer creating direct index
21:59:26.608 [main] INFO o.t.s.indexing.LexiconBuilder - LexiconBuilder active - flushing every 100000 documents, or wh
en memory threshold hit
21:59:27.408 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940102.sgml
21:59:27.686 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940103.sgml
22:02:20.203 [main] INFO o.t.s.i.c.InvertedIndexBuilder - time to traverse direct file: 23.636
22:02:25.540 [main] INFO o.t.s.i.c.InvertedIndexBuilder - time to write inverted file: 5.336
22:02:25.540 [main] INFO o.t.s.i.c.InvertedIndexBuilder - temporary memory used: 388.7 MiB
22:02:25.541 [main] INFO o.t.s.i.c.InvertedIndexBuilder - time to perform one iteration: 30.339
22:02:25.541 [main] INFO o.t.s.i.c.InvertedIndexBuilder - number of pointers processed: 22077489
22:02:25.546 [main] INFO o.t.s.i.c.InvertedIndexBuilder - Finished generating inverted file, rewriting lexicon
22:02:26.991 [main] INFO o.t.s.indexing.LexiconBuilder - Optimising structure lexicon
22:02:26.995 [main] INFO o.t.s.i.FSOMapFileLexiconUtilities - Optimising lexicon with 168704 entries
22:02:27.867 [main] INFO o.t.structures.indexing.Indexer - Finished building the block inverted index...
22:02:27.868 [main] INFO o.t.structures.indexing.Indexer - Time elapsed for inverted file: 32
Total time elapsed: 181 seconds
```

Figura 7-Processo de indexação de dois passos com informação de posição.

Para o processo de indexação de dois passos com informação de posição, obtém-se um tempo de execução de 181 segundos.

De forma contrária, para executar a indexação de dois passos sem informação de posição alterou-se a propriedade *block.indexing* para *False*, e utilizando-se o mesmo comando, como mostra a Figura 8.

```
maryy@Mariana:/mnt/c/Users/maryy/terrier-project-5.5$ ./bin/terrier batchindexing
22:06:54.236 [main] INFO o.terrier.indexing.CollectionFactory - Finished reading collection specification
22:06:54.279 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940101.sgml
22:06:54.551 [main] INFO o.t.structures.indexing.Indexer - creating the data structures ColCHAVE2006stopWstem_1
22:06:54.610 [main] INFO o.t.s.indexing.LexiconBuilder - LexiconBuilder active - flushing every 100000 documents, or wh
en memory threshold hit
22:06:55.351 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940102.sgml
22:06:55.611 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940103.sgml
22:06:55.854 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940104.sgml
22:06:56.162 [main] INFO o.t.i.MultiDocumentFileCollection - TRECCollection 0% processing /mnt/c/users/maryy/Terrier/Co
le_Pub/publico94/ED940105.sgml
22:09:00.577 [main] INFO o.t.s.i.c.InvertedIndexBuilder - Iteration 1 of 1 (estimated) iterations
22:09:13.743 [main] INFO o.t.s.indexing.LexiconBuilder - Optimising structure lexicon
22:09:13.746 [main] INFO o.t.s.i.FSOMapFileLexiconUtilities - Optimising lexicon with 168704 entries
22:09:14.534 [main] INFO o.t.structures.indexing.Indexer - Finished building the inverted index...
22:09:14.534 [main] INFO o.t.structures.indexing.Indexer - Time elapsed for inverted file: 14
Total time elapsed: 140 seconds
```

Figura 8-Processo de indexação de dois passos com informação de posição.

Como mostra a figura, neste caso, obteve-se 140 segundos como tempo de execução.

De forma a comparar os dois processos de indexação e a influência da informação de posição, colocaram-se numa tabela os 4 resultados obtidos:

Tabela 1 – Comparação de tempos de execução dos diferentes processos de indexação.

	Indexação de um passo	Indexação de dois passos
com informação posição	130 s	181 s
sem informação posição	127 s	140 s

Nos dois tipos de indexação testados, verifica-se que o processo sem informação de posição é mais rápido, uma vez que o processo de procura se torna mais rápido. Quando não há informações de posição incluídas no índice, o processo de indexação pode ser mais rápido porque não é necessário analisar a ordem dos termos ou a proximidade entre eles em um documento. Assim, é apenas necessário passar à extração dos termos e registar a sua ocorrência e frequência nos documentos.

Por outro lado, pode tirar-se conclusões também sobre a diferença de tempos entre os processos de indexação de passo único e de dois passos. Geralmente, a indexação de dois passos é mais lenta do que a indexação de um único passo devido à complexidade adicional envolvida e ao aumento do tempo de processamento necessário. Uma vez que na primeira são realizados dois passos, de pré-processamento e indexação, respetivamente, há um maior custo computacional

para leitura, manipulação de dados e indexação e ainda pode existir diferentes atrasos entre as duas etapas. A informação da tabela corrobora as informações anteriores uma vez que se verifica, de facto, o maior tempo de execução nos processos de indexação de dois passos.

2) Vamos usar o Terrier com a coleção Público e o conjunto de interrogações e juízos de relevância disponibilizados no CLEF. Os resultados vão ser obtidos com 50 interrogações

a. Vamos fazer um gráfico de precisão e recall, r-precision, MAP aos 10,20 e 30.

Como passo inicial, utilizou-se o comando *terrier batchretrieval* para recuperar o resultado das queries executadas:

```
maryy@Mariana:/mnt/c/Users/maryy/terrier-project-5.5$ bin/terrier batchretrieval
18:28:17.322 [main] INFO o.t.structures.FSADocumentIndex - Document index requires 417.3 KiB remaining heap is 1.2 GiB
18:28:17.696 [main] INFO o.t.s.BaseCompressingMetaIndex - Structure meta reading lookup file into memory
18:28:17.772 [main] INFO o.t.s.BaseCompressingMetaIndex - Structure meta loading data file into memory
18:28:17.981 [main] INFO o.t.a.batchquerying.TRECQuerying - C251 : medicina alternativa encontrar documentos sobre trat
amentos que empreguem medicina natural ou alternativa aqui sao incluidas terapias como a acupuntura a hemopatia a quirop
ratica entre outras
18:28:17.993 [main] INFO o.t.a.batchquerying.TRECQuerying - Processing query: C251: 'medicina alternativa encontrar doc
umentos sobre tratamentos que empreguem medicina natural ou alternativa aqui sao incluidas terapias como a acupuntura a
hemopatia a quiropratica entre outras'
18:28:17.994 [main] INFO org.terrier.querying.LocalManager - Starting to execute query C251 - medicina alternativa enco
ntrar documentos sobre tratamentos que empreguem medicina natural ou alternativa aqui sao incluidas terapias como a acup
untura a hemopatia a quiropratica entre outras
18:28:18.009 [main] INFO org.terrier.querying.LocalManager - running process TerrierQLParser
18:28:18.042 [main] INFO org.terrier.querying.LocalManager - running process TerrierQLToControls
18:28:23.202 [main] INFO o.t.a.batchquerying.TRECQuerying - Settings of Terrier written to /mnt/c/Users/maryy/terrier-p
roject-5.5/var/results/TF_IDF_0.res.settings
18:28:23.208 [main] INFO o.t.a.batchquerying.TRECQuerying - Finished topics, executed 50 queries in 5.272 seconds, resu
lts written to /mnt/c/Users/maryy/terrier-project-5.5/var/results/TF_IDF_0.res
```

Figura 9- Implementação do comando *batchretrieval*.

Posteriormente, o foi utilizado o comando *terrier evaluate -j*.

```
maryy@Mariana:/mnt/c/Users/maryy/terrier-project-5.5$ bin/terrier batchevaluate -j
18:30:53.059 [main] INFO o.terrier.evaluation.AdhocEvaluation - Evaluating result file: /mnt/c/Users/maryy/terrier-proj
ect-5.5/var/results/TF_IDF_0.res
Average Precision: 0.1750
```

Figura 10- Implementação do comando *batchevaluate -j*.

Posto isto, foi possível obter um ficheiro TF_IDF_0.eval com informações sobre a precisão de cada uma das queries, incluindo outras métricas avaliadas, tais como:

Number of queries = 50

Retrieved = 50000

Relevant = 2904

Relevant retrieved = 1450

Average Precision: 0.1750

R Precision: 0.2234

MAP aos 10: 0.3740;

MAP aos 20: 0.3070;

MAP aos 30: 0.2780.

Os dados relativos ao MAP (Mean Average Precision) obtidos através do programa, foram convertidos para um formato gráfico, apresentado na Figura 11.

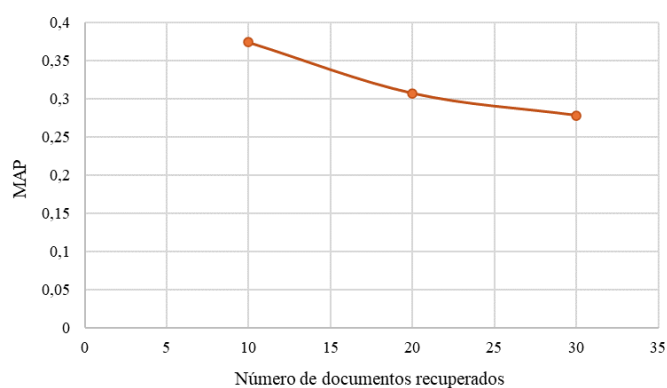


Figura 11- Variação da MAP com o número de documentos recuperados.

Desta forma, com base nos valores obtidos das médias de precisão em cada valor de recall foi possível elaborar um gráfico Precision-Recall para o modelo TD_IDF, apresentado na Figura 12.

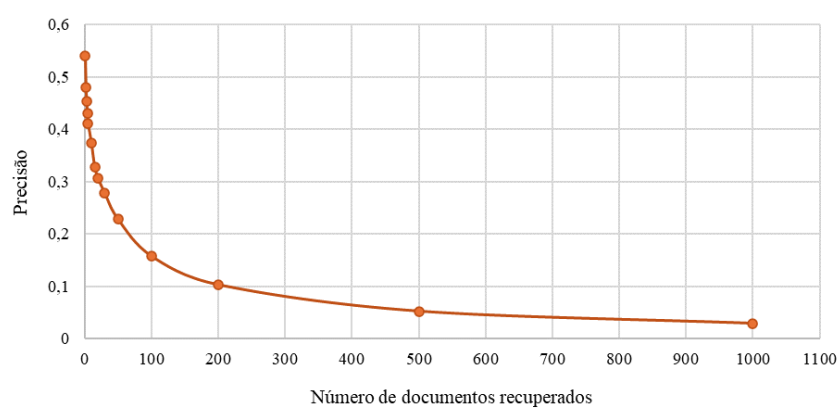


Figura 12- Variação da precisão conforme o número de documentos recuperados.

b. Vamos compara dois métodos de ponderação disponibilizados no Terrier! (por exemplo! TF-IDF e BM25) e em particular um baseado no DFR.

Foram então, averiguados os métodos de ponderação TF-IDF, apresentado anteriormente, assim como BM5 e DLH, sendo o último baseado em DFR.

Recorrendo ao modelo BM25, obtiveram-se os seguintes dados:

Number of queries = 50
Retrieved = 50000
Relevant = 2904
Relevant retrieved = 1464
Average Precision: 0.1756
R Precision: 0.2277
MAP aos 10: 0.3760;
MAP aos 20: 0.3040;
MAP aos 30: 0.2747.

Relativamente ao modelo DLH, foram obtidas as seguintes informações:

Number of queries = 50
Retrieved = 50000
Relevant = 2904
Relevant retrieved = 1407
Average Precision: 0.1637
R Precision: 0.2098
MAP aos 10: 0.3540;
MAP aos 20: 0.2880;
MAP aos 30: 0.2473.

A partir dos dados de precisão e MAP obtidos por número de documentos recuperados para cada modelo foram traçados dois gráficos, um para cada uma das métricas (Figuras 13 e 14).

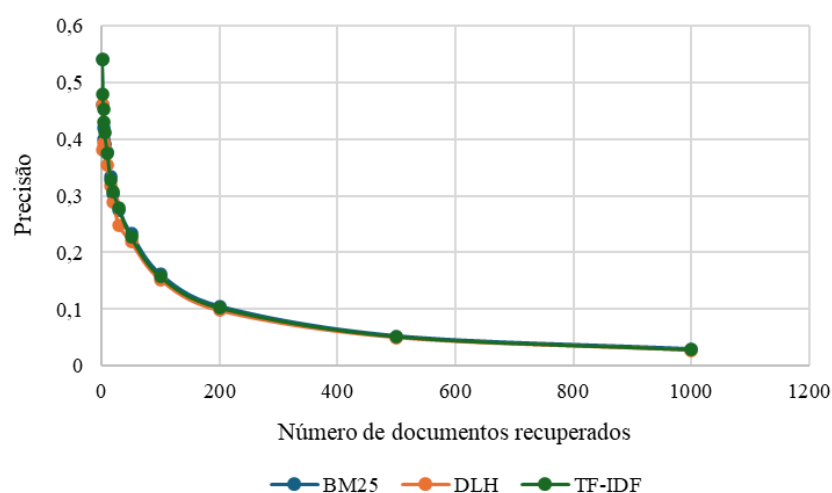


Figura 13- Variação da precisão em relação ao número de documentos recuperados para cada modelo.

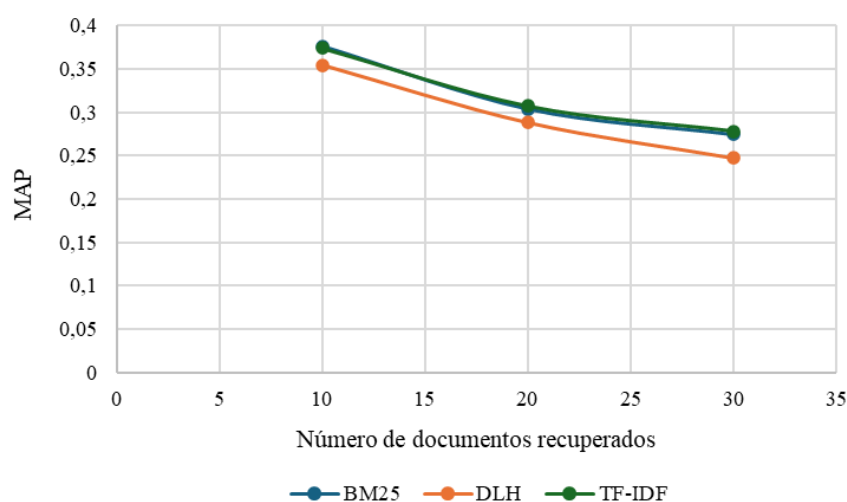


Figura 14- Variação da MAP com o número de documentos recuperados para cada modelo.

Ao observar estes resultados, conclui-se que em termos de precisão, os três modelos têm comportamentos muito semelhantes, porém, quanto à MAP, o modelo DLH tem um desempenho ligeiramente mais baixo. Isto poderá estar relacionado com o facto de o modelo DHR ser uma abordagem mais sofisticada. Ou seja, embora os modelos possam recuperar um número semelhante de documentos relevantes, a forma como o modelo DLH calcula a relevância pode resultar numa distribuição de relevância dos documentos diferente dos restantes modelos, afetando a métrica MAP.

Tal relação pode ser ainda averiguada na Tabela 2.

Tabela 2- Valores de precisão média para os três modelos analisados.

Métodos	TF-IDF	BM25	DLH
Average Precision	0,1750	0,1756	0,1637

c. Qual é a melhoria dos resultados se usarmos expansão de interrogações?

A expansão de interrogações é um processo que retira os termos mais relevantes dos documentos mais importantes que são devolvidos como resultado da pesquisa. Desta forma, a expansão de *queries* tem como objetivo melhorar a precisão e a qualidade dos resultados.

De forma a verificar a influência da expansão de interrogações na melhoria dos mesmos, foram comparados os resultados de *Average Precision* com e sem a utilização da expansão de *queries*.

Para a obtenção dos resultados com a expansão de interrogações, foi utilizado o comando `bin/terrier batchretrieval -q` para a recuperação da coleção teste indexada. De seguida, utilizou-se o comando `bin/terrier evaluate -j` para a avaliação e consequente cálculo da *Average Precision*. Obteve-se então uma *Average Precision* de 0.1750.

No que diz respeito à obtenção dos resultados sem expansão de interrogações, foi utilizado o comando `bin/terrier batchretrieval` seguido do comando `bin/terrier evaluate -j`. O resultado obtido foi uma *Average Precision* de 0.1750.

Os resultados demonstram que, em ambos os casos, a *Average Precision* foi igual a 0,1750. Esta observação sugere que a expansão de interrogações não teve um impacto mensurável na precisão dos resultados neste contexto. Uma possível explicação para este fenómeno poderá ser o tamanho limitado ou a natureza da coleção de documentos utilizada, que poderá não proporcionar benefícios significativos com a expansão de *queries*.

d. O facto de se usar stopwords e stemming melhora os resultados?

Com o intuito de analisar o impacto da utilização de *stopwords* e *stemming* nos resultados de recuperação de informação, foram conduzidos diversos testes utilizando o Terrier, cujos resultados estão apresentados na Tabela 3. É importante destacar que, para viabilizar essa análise, a propriedade *termpipelines* foi configurada no ficheiro *terrier.properties*.

Tabela 3- Influência das *stopwords* e do *stemming* na *Average Precision*.

Utilização	Configuração Terrier	Average Precision
StopWords+Stemming	termpipelines= Stopwords,PortugueseSnowballStemmer	0,1750
StopWords	termpipelines= Stopwords	0,0708
Stemming	termpipelines= PortugueseSnowballStemmer	0,1708
Nenhuma Técnica	termpipelines=	0,0578

A partir da análise da Tabela 3, é possível inferir que a utilização combinada de *stopwords* e de *stemming* obteve a maior *Average Precision*, alcançando 0,1750. Isto sugere que a combinação de ambas as técnicas contribuiu para uma melhor precisão na recuperação de informação. Por outro lado, quando foram utilizadas apenas *stopwords*, a *Average Precision* foi significativamente menor (0,0708). Da mesma forma, ao aplicar apenas *stemming*, a *Average Precision* foi ligeiramente inferior em comparação com *StopWords+ Stemming*, obtendo-se 0,1708. Finalmente, quando nenhuma das técnicas foi aplicada, a *Average Precision* foi a mais baixa, com 0,0578.

Com base nos resultados apresentados, pode concluir-se que a utilização de *stopwords* e *stemming* contribui para a melhoria dos resultados de recuperação de informação. Este facto é expectável, visto que a remoção de *stopwords* reduz o ruído nos documentos, concentrando-se nos termos mais informativos, enquanto o *stemming* normaliza as palavras, agrupando variações morfológicas e aumentando a correspondência entre os termos de consulta e os termos nos documentos indexados.

Conclusão

Em conclusão, a experiência com a ferramenta Terrier proporcionou um melhor entendimento do processo de recuperação de informações. Ao longo deste trabalho prático, foram explorados diversos aspetos, desde a indexação até a realização de consultas e recuperação de informação.

Compreendeu-se que a indexação desempenha um papel fundamental na eficiência da recuperação de informações, permitindo a organização e processamento eficaz dos documentos para facilitar consultas rápidas e precisas. Utilizando o Terrier, foi possível criar índices robustos que possibilitaram lidar com grandes conjuntos de dados de forma eficiente.

Além disso, ganharam-se habilidades na formulação e execução de consultas, incluindo o uso de recursos avançados como consultas compostas e expansão de interrogações para melhorar os resultados da recuperação. A capacidade de ajustar parâmetros e avaliar o desempenho das consultas permitiu refinamento das estratégias e obtenção de resultados mais relevantes.

O recurso de recuperação em *batch* revelou-se igualmente importante, automatizando o processo de consulta e facilitando a avaliação do desempenho do sistema em grande escala. Ao testar várias opções de indexação passo único e de dois passos, bem como diferentes algoritmos de ponderação, concluiu-se sobre a eficácia do sistema em lidar com conjuntos de dados diversos e em constante evolução. Essa abordagem sistemática permitiu uma comparação detalhada e uma compreensão mais profunda das melhores práticas em processos de recuperação de informação.

Em resumo, esta experiência proporcionou à equipe uma compreensão aprofundada dos desafios e oportunidades associados à recuperação de informação e ao uso de ferramentas como o Terrier.

Bibliografia:

- [1] GitHub - terrier-org/terrier-core: Terrier IR Platform. (s.d.). GitHub: Disponível em: <https://github.com/terrier-org/terrier-core>
- [2] Ramadhan, Mazen & Latip, Rohaya & Hussin, Masnida & Abdulkarem, Mohammed. (2016). Indexing Strategies of MapReduce for Information Retrieval in Big Data. Journal of Computer Science and Technology. Disponível em: https://www.researchgate.net/publication/320765042_Indexing_Strategies_of_MapReduce_for_Information_Retrieval_in_Big_Data
- [3] Apache Hadoop 3.3.6 – MapReduce Tutorial. (s.d.). Apache Hadoop. Disponível em: <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-clientcore/MapReduceTutorial.htm>
- [4] MIT - Massachusetts Institute of Technology. Disponível em: <https://web.mit.edu/6.033/2004/wwwdocs/papers/page98pagerank.pdf>